Spring Data JPA Count Query

https://gemini.google.com/app/00d8b10bf3a41a23

User prompt: INSERT INTO redemption (voucher_id, campaign_id, user_id, created_at) VALUES (4, 1, 'user-99', '2026-06-01 10:15:00'), (5, 1, 'user-99', '2026-06-02 09:40:00'); this my data jpa find count by given campaign_id and user_id

Response: To count the number of redemptions for a given `campaignId` and `userId` using Spring Data JPA, you can use a derived query method or an explicit `@Query` annotation in your repository.

Here is how you can implement it:

1. Entity Definition (`Redemption.java`)

Assuming your entity looks something like this:

```
@Entity
@Table(name = "redemption")
public class Redemption {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "voucher_id")
    private Long voucherId;

    @Column(name = "campaign_id")
    private Long campaignId; // Or int/String depending on your schema

    @Column(name = "user_id")
    private String userId;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // Getters, setters, constructors...
}
```

2. Repository Method (`RedemptionRepository.java`)

You have two clean options to define this in your repository interface:

Option A: Derived Count Query (Recommended)

Spring Data JPA automatically parses the method name and constructs the query for you.

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface RedemptionRepository extends JpaRepository<Redemption, Long> {

    // Derived query to count rows matching both campaignId and userId
    long countByCampaignIdAndUserId(Long campaignId, String userId);
}
```

Option B: Explicit JPQL Query

If you prefer explicit control or want to write custom JPQL/SQL, use the `@Query` annotation:

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

@Repository
public interface RedemptionRepository extends JpaRepository<Redemption, Long> {

    @Query("SELECT COUNT(r) FROM Redemption r WHERE r.campaignId = :campaignId AND r.userId = :userId")
    long countRedemptions(@Param("campaignId") Long campaignId, @Param("userId") String userId);
}
```

3. Usage in a Service

You can inject and call the repository method like this:

```
@Service
public class RedemptionService {

    @Autowired
    private RedemptionRepository redemptionRepository;
```

```
    public void checkUserRedemptions() {
        long count = redemptionRepository.countByCampaignIdAndUserId(1L, "user-99");
        System.out.println("Total redemptions: " + count); // Output will be 2 based on your sample data
    }
}
```

Are you working with Spring Boot 3.x, and do you need help setting up pagination or checking if this count exceeds a specific limit (e.g., maximum redemptions per user)?

User prompt: INSERT INTO campaign (id, name, client_code, total_stock, remaining_stock, active) VALUES (1, 'Raya Flash Deal', 'ACME', 100, 100, TRUE), (2, 'Merdeka Giveaway', 'NOVA', 50, 50, TRUE), (3, 'Expired Test Campaign', 'ACME', 10, 0, FALSE); INSERT INTO voucher (id, campaign_id, code, status, redeemed_by, redeemed_at) VALUES (1, 1, 'RAYA-0001', 'ACTIVE', NULL, NULL), (2, 1, 'RAYA-0002', 'ACTIVE', NULL, NULL), (3, 1, 'RAYA-0003', 'ACTIVE', NULL, NULL), (4, 1, 'RAYA-0004', 'REDEEMED', 'user-99', '2026-06-01 10:15:00'), (5, 1, 'RAYA-0005', 'VOID', 'user-99', '2026-06-02 09:40:00'), (6, 1, 'RAYA-0006', 'ACTIVE', NULL, NULL), (7, 2, 'MRDK-0001', 'ACTIVE', NULL, NULL), (8, 2, 'MRDK-0002', 'ACTIVE', NULL, NULL), (9, 3, 'EXPD-0001', 'ACTIVE', NULL, NULL); INSERT INTO redemption (voucher_id, campaign_id, user_id, created_at) VALUES (4, 1, 'user-99', '2026-06-01 10:15:00'), (5, 1, 'user-99', '2026-06-02 09:40:00'); DROP TABLE IF EXISTS redemption; DROP TABLE IF EXISTS voucher; DROP TABLE IF EXISTS campaign; CREATE TABLE campaign (id BIGINT PRIMARY KEY, name VARCHAR(200) NOT NULL, client_code VARCHAR(50) NOT NULL, total_stock INT NOT NULL, remaining_stock INT NOT NULL, active BOOLEAN NOT NULL DEFAULT TRUE, user_voucher_limit INT NOT NULL DEFAULT 2); CREATE TABLE voucher (id BIGINT PRIMARY KEY, campaign_id BIGINT NOT NULL, code VARCHAR(40) NOT NULL, status VARCHAR(20) NOT NULL, redeemed_by VARCHAR(80), redeemed_at TIMESTAMP); CREATE TABLE redemption (id BIGINT AUTO_INCREMENT PRIMARY KEY, voucher_id BIGINT NOT NULL, campaign_id BIGINT NOT NULL, user_id VARCHAR(80) NOT NULL, created_at TIMESTAMP NOT NULL); public RedeemResponse redeem(String code, String userId) { Voucher voucher = voucherRepository.findByCode(code); if (voucher == null) { return RedeemResponse.fail("Voucher not found"); } if ("REDEEMED".equals(voucher.getStatus())) { return RedeemResponse.fail("Voucher already redeemed"); } if ("VOID".equals(voucher.getStatus())) { return RedeemResponse.fail("Voucher is void"); } Campaign campaign = campaignRepository.findById(voucher.getCampaignId()).get(); if (!campaign.isActive()) { return RedeemResponse.fail("Campaign is not active"); } if (campaign.getRemainingStock() <= 0) { return RedeemResponse.fail("Campaign out of stock"); } int userRedemptionCount = voucherRepository.countByCampaignIdAndRedeemedByAndStatus(campaign.getId(), userId, "REDEEMED"); if(userRedemptionCount >= campaign.getUserVoucherLimit()){ return RedeemResponse.fail("All vouchers have been redeemed for this campaign"); } voucher.setStatus("REDEEMED"); voucher.setRedeemedBy(userId); voucher.setRedeemedAt(new Date()); voucherRepository.save(voucher); campaign.setRemainingStock(campaign.getRemainingStock() - 1); campaignRepository.save(campaign); Redemption redemption = new Redemption(voucher.getId(), campaign.getId(), userId, new Date()); redemptionRepository.save(redemption); try { auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId); } catch (Exception e) { log.error("audit call failed"); } return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock()); } consider multiple pod for call this method same time, how to solve

Response: When running multiple pods simultaneously, your current code suffers from **race conditions**. Two pods can read the voucher status as `ACTIVE` or check the user's redemption count at the exact same millisecond, allowing a user to bypass limits or redeem the same voucher twice.

To solve this across distributed pods, you need to rely on **database-level atomicity and locking** rather than in-memory checks.

Solution 1: Atomic Voucher Update (Prevents Double Spending)

Instead of checking the status in Java (`if ("REDEEMED".equals(...))`), perform a conditional update directly in the database. If another pod updates the voucher first, this query will affect 0 rows.

Add this method to your `VoucherRepository`:

```
@Modifying
@Query("UPDATE Voucher v SET v.status = 'REDEEMED', v.redeemedBy = :userId, v.redeemedAt = :redeemedAt WHE
int claimVoucher(@Param("voucherId") Long voucherId, @Param("userId") String userId, @Param("redeemedAt")
```

In your service, check the return value of `claimVoucher()`. If it returns `0`, it means another pod already claimed it:

```
int updatedRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
if (updatedRows == 0) {
    return RedeemResponse.fail("Voucher already redeemed or invalid");
}
```

- **Verification:** Run a load test with multiple threads trying to redeem the exact same voucher code concurrently; only one request should succeed while the rest fail.

Solution 2: Pessimistic Locking for Campaign Stock

To prevent multiple pods from decrementing `remaining_stock` below zero simultaneously, lock the campaign row during the read operation using Spring Data JPA's `@Lock`.

Update your `CampaignRepository`:

```
import jakarta.persistence.LockModeType;
import org.springframework.data.jpa.repository.Lock;

@Repository
public interface CampaignRepository extends JpaRepository<Campaign, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT c FROM Campaign c WHERE c.id = :id")
    Optional<Campaign> findByIdWithLock(@Param("id") Long id);
}
```

Use `findByIdWithLock` in your service instead of standard `findById`:

```
Campaign campaign = campaignRepository.findByIdWithLock(voucher.getCampaignId())
    .orElseThrow(() -> new RuntimeException("Campaign not found"));
```

- **Verification:** Check database logs to ensure `SELECT ... FOR UPDATE` is executed, locking the campaign row until the transaction commits.

Solution 3: Unique Database Constraint for User Limits

To strictly enforce that a user cannot exceed `user_voucher_limit` for a campaign even under high concurrency, add a unique constraint or handle it via database uniqueness if applicable, or rely on a transactional unique check with the `redemption` table.

Since you insert a record into the `redemption` table every time:

```
CREATE TABLE redemption (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    voucher_id  BIGINT      NOT NULL UNIQUE, -- Ensures a voucher can only have 1 redemption
    campaign_id BIGINT      NOT NULL,
    user_id     VARCHAR(80) NOT NULL,
    created_at  TIMESTAMP   NOT NULL
);
```

By adding a `UNIQUE` constraint on `voucher_id` in the `redemption` table, the database will automatically throw a constraint violation exception if two pods try to insert a redemption for the same voucher simultaneously.

User prompt: for handle update voucher and redemption, there can be multiple user redeem same voucher how to prevent

Response: To completely prevent multiple users from successfully claiming the **exact same voucher**, you need a two-layered defense: **Database Constraints** combined with **Atomic Conditional Updates**.

Even if two different users hit your API at the exact same millisecond across different pods, only **one** request will win the race.

Step 1: Add a Unique Constraint to the `redemption` Table

A voucher should only ever be redeemed once. Enforce this at the database level by making `voucher_id` unique in the `redemption` table.

```
ALTER TABLE redemption ADD CONSTRAINT uk_voucher_id UNIQUE (voucher_id);
```

- **Why:** If two pods somehow bypass application logic and try to insert a redemption record for the same `voucher_id` at the same time, the database will throw a `DataIntegrityViolationException` on the second insert, rolling back the transaction.

Step 2: Use an Atomic Conditional Update

Instead of checking `if ("ACTIVE".equals(voucher.getStatus()))` in Java (which causes race conditions), let the database do the safety check atomically.

Update your `VoucherRepository` with a conditional update query:

```
@Modifying
@Query("UPDATE Voucher v SET v.status = 'REDEEMED', v.redeemedBy = :userId, v.redeemedAt = :redeemedAt WHE
int claimVoucher(@Param("voucherId") Long voucherId, @Param("userId") String userId, @Param("redeemedAt")
```

- **Why:** The `WHERE v.status = 'ACTIVE'` clause ensures that if User A updates the voucher first, its status changes to `REDEEMED`. When User B's request hits a millisecond later, the database finds `0` rows matching `status = 'ACTIVE'` and updates nothing.

Step 3: Implement it safely inside a `@Transactional` Service

Wrap your redemption logic in a transaction so that if any step fails (or if the atomic update affects 0 rows), everything rolls back cleanly.

```
@Transactional
public RedeemResponse redeem(String code, String userId) {
    // 1. Fetch voucher (basic check to see if it exists at all)
    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) {
        return RedeemResponse.fail("Voucher not found");
    }

    // 2. ATOMIC CLAIM: Try to claim the voucher only if it's currently ACTIVE.
    // If another user/pod claimed it first, updatedRows will be 0.
    int updatedRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
    if (updatedRows == 0) {
        return RedeemResponse.fail("Voucher has already been redeemed or is invalid");
    }

    // 3. Fetch and lock campaign stock
    Campaign campaign = campaignRepository.findByIdWithLock(voucher.getCampaignId())
        .orElseThrow(() -> new RuntimeException("Campaign not found"));

    if (!campaign.isActive()) {
        throw new RuntimeException("Campaign is not active"); // Triggers rollback of the voucher claim
    }

    if (campaign.getRemainingStock() <= 0) {
```

```
            throw new RuntimeException("Campaign out of stock"); // Triggers rollback
        }

        // 4. Check user limits per campaign
        long userRedemptionCount = redemptionRepository.countByCampaignIdAndUserId(campaign.getId(), userId);
        if (userRedemptionCount >= campaign.getUserVoucherLimit()) {
            throw new RuntimeException("User limit reached for this campaign"); // Triggers rollback
        }

        // 5. Decrement campaign stock
        campaign.setRemainingStock(campaign.getRemainingStock() - 1);
        campaignRepository.save(campaign);

        // 6. Save redemption record (Database UNIQUE constraint on voucher_id protects this too)
        try {
            Redemption redemption = new Redemption(voucher.getId(), campaign.getId(), userId, new Date());
            redemptionRepository.save(redemption);
        } catch (DataIntegrityViolationException e) {
            return RedeemResponse.fail("Voucher has already been redeemed");
        }

        // 7. Audit log (outside critical path or handled asynchronously)
        try {
            auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId);
        } catch (Exception e) {
            log.error("audit call failed");
        }

        return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock());
    }
```

Summary of How This Prevents Double Redemption:

1. **Pod 1 & Pod 2** both try to call `claimVoucher()` at the exact same time.
2. The database processes them sequentially. **Pod 1** successfully updates the row (`updatedRows = 1`). **Pod 2** gets `updatedRows = 0` and is immediately rejected with `"Voucher has already been redeemed"`.
3. Even if they somehow passed that, the **`UNIQUE` constraint** on the `redemption` table's `voucher_id` acts as a final safety net, throwing an exception and rolling back the transaction.

User prompt: // 4. Check user limits per campaign long userRedemptionCount = redemptionRepository.countByCampaignIdAndUserId(campaign.getId(), userId); if (userRedemptionCount >= campaign.getUserVoucherLimit()) { throw new RuntimeException("User limit reached for this campaign"); // Triggers rollback } Each campaign should define the maximum number of vouchers a single user may redeem in that campaign. When a user tries to redeem beyond their limit, the request should be rejected with a clear reason. Use `2` as the default limit for existing campaigns. void also consider use and reached limit?

Response: If a voucher or redemption is marked as **`VOID`** (meaning it was cancelled, revoked, or invalidated), it **should not count** toward the user's limit. The user should still be allowed to redeem a valid voucher up to their campaign limit.

How this is handled depends on whether you count redemptions via the `voucher` table or the `redemption` table:

Option 1: Counting via the `voucher` Table (Recommended)

Your original code already handled this nicely by explicitly checking for the `REDEEMED` status and filtering out `VOID` or `ACTIVE` vouchers:

```
int userRedemptionCount = voucherRepository.countByCampaignIdAndRedeemedByAndStatus(
    campaign.getId(),
    userId,
    "REDEEMED"
);
```

- **Why this works:** Since voucher `RAYA-0005` has a status of `VOID`, it is completely ignored by this count. If a user has 1 valid redemption and 1 voided redemption, their count will be `1`, allowing them to redeem one more if their limit is `2`.

Option 2: Counting via the `redemption` Table (With VOID Exclusion)

If you prefer to track limits using your `redemption` table, you need to ensure your query ignores redemptions linked to a `VOID` voucher by joining with the `voucher` table:

Repository Method (`RedemptionRepository.java`):

```
@Query("""
    SELECT COUNT(r) FROM Redemption r
    JOIN Voucher v ON r.voucherId = v.id
    WHERE r.campaignId = :campaignId
      AND r.userId = :userId
      AND v.status <> 'VOID'
""")
long countValidRedemptions(@Param("campaignId") Long campaignId, @Param("userId") String userId);
```

- **Why this works:** It explicitly checks that the voucher's status is **not** `VOID`. If a voucher gets voided later, its redemption won't block the user from making a new valid redemption.

Summary Rule for `VOID`

- **Active/Redeemed:** Counts toward the limit.
- **Void:** Excluded from the limit (frees up a slot for the user).

User prompt: int userRedemptionCount = voucherRepository.countByCampaignIdAndRedeemedByAndStatus(campaign.getId(), userId, "REDEEMED"); if same multiple request, will have race conditions

Response: You are entirely correct. Spotting that is crucial.

Even with the campaign locked, your code has a classic **Check-Then-Act (TOCTOU - Time-of-Check to Time-of-Use) race condition**.

If a user fires two requests simultaneously across different pods:

1. **Pod 1** checks the count -> finds `1` redemption (limit is 2).
2. **Pod 2** checks the count *at the exact same millisecond* -> also finds `1` redemption.
3. Both pods pass the check (`1 < 2`) and both successfully redeem, resulting in `3` total redemptions for the user, completely bypassing the limit.

Because the check and the write happen in two separate steps, database constraints alone cannot easily protect a "maximum count of N" unless you use specific concurrency controls.

Two industry-standard ways solve this across multiple pods:

Solution 1: Distributed Lock (Redis / Redisson) — *Recommended for Multi-Pod*

The cleanest way to handle user-specific limits across multiple pods is to use a **Distributed Lock** keyed by the user and campaign. This ensures that concurrent requests from the *same user* are forced to run sequentially.

If you use Spring Boot with Redis, you can use **Redisson**:

```
@Autowired
private RedissonClient redissonClient;

public RedeemResponse redeem(String code, String userId) {
    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) {
        return RedeemResponse.fail("Voucher not found");
    }

    Campaign campaign = campaignRepository.findById(voucher.getCampaignId())
        .orElseThrow(() -> new RuntimeException("Campaign not found"));

    // Create a unique lock key for this user and campaign
    String lockKey = "lock:redeem:" + campaign.getId() + ":" + userId;
    RLock lock = redissonClient.getLock(lockKey);

    try {
        // Try to acquire lock (wait up to 3 seconds, hold for up to 5 seconds)
        boolean isLocked = lock.tryLock(3, 5, TimeUnit.SECONDS);
        if (!isLocked) {
            return RedeemResponse.fail("Too many concurrent requests. Please try again.");
        }

        // --- CRITICAL SECTION (Runs sequentially for this user) ---

        // 1. Re-check user limit inside the lock
        int userRedemptionCount = voucherRepository.countByCampaignIdAndRedeemedByAndStatus(
            campaign.getId(), userId, "REDEEMED"
        );
        if (userRedemptionCount >= campaign.getUserVoucherLimit()) {
            return RedeemResponse.fail("All vouchers have been redeemed for this campaign");
        }

        // 2. Atomic claim voucher
        int updatedRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
        if (updatedRows == 0) {
            return RedeemResponse.fail("Voucher already redeemed or invalid");
        }

        // 3. Update campaign stock and save redemption...
        campaign.setRemainingStock(campaign.getRemainingStock() - 1);
        campaignRepository.save(campaign);

        Redemption redemption = new Redemption(voucher.getId(), campaign.getId(), userId, new Date());
        redemptionRepository.save(redemption);
```

```
        return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock());

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return RedeemResponse.fail("Redemption interrupted");
    } finally {
        // Always release the lock
        if (lock.isHeldByCurrentThread()) {
            lock.unlock();
        }
    }
}
```

Solution 2: Database Advisory Locks (If you don't use Redis)

If your architecture doesn't use Redis and relies purely on a relational database (like PostgreSQL or MySQL), you can acquire a database-level **advisory lock** using a hash of the user ID and campaign ID.

For example, in MySQL:

```
SELECT GET_LOCK(:lockName, 5); -- Waits up to 5 seconds for a lock
-- Run your count check and redemption logic here
SELECT RELEASE_LOCK(:lockName);
```

This forces any concurrent requests for the exact same user/campaign combination to queue up sequentially at the database level, preventing the race condition entirely.

User prompt: better approach for this limit check

Response: When dealing with a dynamic per-user limit (e.g., max N vouchers per user) across multiple pods, preventing race conditions cleanly depends on your infrastructure.

Two industry-standard approaches stand out as the best, depending on whether you want a **pure database solution** or a **high-scale distributed cache solution**.

Approach 1: The Database-Native Solution (User-Campaign Tracking Table + Row Lock)

Instead of running a heavy `COUNT(*)` query across the `redemption` table every time (which is slow and hard to lock safely), introduce a dedicated tracking table or lock a user-campaign record using **Pessimistic Locking (`FOR UPDATE`)**.

1. Create a tracking table:

```
CREATE TABLE user_campaign_quota (
    user_id     VARCHAR(80) NOT NULL,
    campaign_id BIGINT      NOT NULL,
    used_count  INT         NOT NULL DEFAULT 0,
    PRIMARY KEY (user_id, campaign_id)
);
```

2. Implement the check safely in your service:

By locking the specific row for that user and campaign, any concurrent request from the same user on another pod is forced to **wait** until the first transaction finishes.

```
@Transactional
public RedeemResponse redeem(String code, String userId) {
    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) return RedeemResponse.fail("Voucher not found");

    Campaign campaign = campaignRepository.findById(voucher.getCampaignId())
        .orElseThrow(() -> new RuntimeException("Campaign not found"));

    // 1. Lock or initialize the user's quota row for this campaign
    // This serializes concurrent requests from the SAME user across pods
    UserCampaignQuota quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
        .orElseGet(() -> quotaRepository.save(new UserCampaignQuota(userId, campaign.getId(), 0)));

    // 2. Check limit atomically inside the lock
    if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) {
        return RedeemResponse.fail("User limit reached for this campaign");
    }

    // 3. Atomically claim the voucher
    int updatedRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
    if (updatedRows == 0) {
        return RedeemResponse.fail("Voucher already redeemed");
    }

    // 4. Increment quota and save
```

```
    quota.setUsedCount(quota.getUsedCount() + 1);
    quotaRepository.save(quota);

    // 5. Update campaign stock & save redemption record...
    // (Same as before)
}
```

- **Why it's great:** No extra infrastructure (like Redis) is required. It's 100% atomic, ACID-compliant, and automatically handles the limit check without race conditions.

Approach 2: The High-Scale Distributed Solution (Redis Distributed Lock)

If your system experiences massive traffic spikes (e.g., flash sales where thousands of requests hit simultaneously), locking database rows can cause connection pool exhaustion and database bottlenecks.

Using **Redis (Redisson)** to lock only the specific user-campaign key keeps the database light:

```
String lockKey = "lock:quota:" + campaign.getId() + ":" + userId;
RLock lock = redissonClient.getLock(lockKey);

try {
    // Force concurrent requests from the same user to wait in line
    if (!lock.tryLock(3, 5, TimeUnit.SECONDS)) {
        return RedeemResponse.fail("Too many concurrent requests. Please try again.");
    }

    // Safe to check count and redeem here without race conditions
    int userRedemptionCount = voucherRepository.countByCampaignIdAndRedeemedByAndStatus(
        campaign.getId(), userId, "REDEEMED"
    );
    if (userRedemptionCount >= campaign.getUserVoucherLimit()) {
        return RedeemResponse.fail("User limit reached");
    }

    // Proceed with voucher claim...

} finally {
    if (lock.isHeldByCurrentThread()) {
        lock.unlock();
    }
}
```

- **Why it's great:** It protects your database from being hammered by concurrent limit-checking queries from the same user.

Summary Recommendation

- **Choose Approach 1 (Database Quota + Row Lock)** if you want a clean, reliable, database-centric solution without adding caching infrastructure.
- **Choose Approach 2 (Redis Lock)** if you are building a high-throughput flash sale system where database connection pooling is a bottleneck.

User prompt: approach 1 show me quotaRepository and modela

Response: Here is the complete implementation for **Approach 1**, including the model, repository, and service integration.

1. Model: `UserCampaignQuota.java`

Create an entity that represents a composite key of `userId` and `campaignId`.

```
package com.example.redemption.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.IdClass;
import jakarta.persistence.Table;

@Entity
@Table(name = "user_campaign_quota")
@IdClass(UserCampaignQuotaId.class)
public class UserCampaignQuota {

    @Id
    @Column(name = "user_id", length = 80, nullable = false)
    private String userId;

    @Id
    @Column(name = "campaign_id", nullable = false)
    private Long campaignId;
```

```
    @Column(name = "used_count", nullable = false)
    private int usedCount;

    public UserCampaignQuota() {}

    public UserCampaignQuota(String userId, Long campaignId, int usedCount) {
        this.userId = userId;
        this.campaignId = campaignId;
        this.usedCount = usedCount;
    }

    public String getUserId() {
        return userId;
    }

    public void setUserId(String userId) {
        this.userId = userId;
    }

    public Long getCampaignId() {
        return campaignId;
    }

    public void setCampaignId(Long campaignId) {
        this.campaignId = campaignId;
    }

    public int getUsedCount() {
        return usedCount;
    }

    public void setUsedCount(int usedCount) {
        this.usedCount = usedCount;
    }
}
```

Composite Key Primary Class: `UserCampaignQuotaId.java`

```
package com.example.redemption.model;

import java.io.Serializable;
import java.util.Objects;

public class UserCampaignQuotaId implements Serializable {

    private String userId;
    private Long campaignId;

    public UserCampaignQuotaId() {}

    public UserCampaignQuotaId(String userId, Long campaignId) {
        this.userId = userId;
        this.campaignId = campaignId;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        UserCampaignQuotaId that = (UserCampaignQuotaId) o;
        return Objects.equals(userId, that.userId) && Objects.equals(campaignId, that.campaignId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(userId, campaignId);
    }
}
```

2. Repository: `QuotaRepository.java`

Use JPA's `@Lock(LockModeType.PESSIMISTIC_WRITE)` to execute `SELECT ... FOR UPDATE` when retrieving the record.

```
package com.example.redemption.repository;

import com.example.redemption.model.UserCampaignQuota;
import com.example.redemption.model.UserCampaignQuotaId;
import jakarta.persistence.LockModeType;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
```

```
import org.springframework.stereotype.Repository;

@Repository
public interface QuotaRepository extends JpaRepository<UserCampaignQuota, UserCampaignQuotaId> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT q FROM UserCampaignQuota q WHERE q.userId = :userId AND q.campaignId = :campaignId")
    Optional<UserCampaignQuota> findByUserIdAndCampaignIdWithLock(
        @Param("userId") String userId,
        @Param("campaignId") Long campaignId
    );
}
```

3. Database Table DDL

Be sure to add this table definition to your SQL migrations:

```
CREATE TABLE user_campaign_quota (
    user_id     VARCHAR(80) NOT NULL,
    campaign_id BIGINT      NOT NULL,
    used_count  INT         NOT NULL DEFAULT 0,
    PRIMARY KEY (user_id, campaign_id)
);
```

4. Updated `redeem` Method Usage

```
@Transactional
public RedeemResponse redeem(String code, String userId) {

    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) {
        return RedeemResponse.fail("Voucher not found");
    }

    Campaign campaign = campaignRepository.findById(voucher.getCampaignId())
        .orElseThrow(() -> new RuntimeException("Campaign not found"));

    if (!campaign.isActive()) {
        return RedeemResponse.fail("Campaign is not active");
    }

    if (campaign.getRemainingStock() <= 0) {
        return RedeemResponse.fail("Campaign out of stock");
    }

    // 1. Lock/Fetch or initialize user's quota for this campaign
    UserCampaignQuota quota = quotaRepository
        .findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
        .orElseGet(() -> quotaRepository.save(new UserCampaignQuota(userId, campaign.getId(), 0)));

    // 2. Check quota limit
    if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) {
        return RedeemResponse.fail("All vouchers have been redeemed for this campaign");
    }

    // 3. Atomically update voucher status
    int updatedRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
    if (updatedRows == 0) {
        return RedeemResponse.fail("Voucher already redeemed or invalid");
    }

    // 4. Increment and save quota
    quota.setUsedCount(quota.getUsedCount() + 1);
    quotaRepository.save(quota);

    // 5. Update campaign stock
    campaign.setRemainingStock(campaign.getRemainingStock() - 1);
    campaignRepository.save(campaign);

    // 6. Save redemption history
    Redemption redemption = new Redemption(voucher.getId(), campaign.getId(), userId, new Date());
    redemptionRepository.save(redemption);

    // Audit log
    try {
        auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId);
    } catch (Exception e) {
        log.error("audit call failed", e);
    }

    return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock());
}
```

User prompt: @Transactional public RedeemResponse redeem(String code, String userId) { Voucher voucher = voucherRepository.findByCode(code); if (voucher == null) { return RedeemResponse.fail("Voucher not found"); } int updatedRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date()); if (updatedRows == 0) { return RedeemResponse.fail("Voucher has already been redeemed or is invalid"); } Campaign campaign = campaignRepository.findByIdWithLock(voucher.getCampaignId()) .orElseThrow(() -> new RuntimeException("Campaign not found")); UserCampaignQuota quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId()) .orElseGet(() -> quotaRepository.save(new UserCampaignQuota(userId, campaign.getId(), 0))); if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) { return RedeemResponse.fail("User limit reached for this campaign"); } if (!campaign.isActive()) { throw new RuntimeException("Campaign is not active"); } if (campaign.getRemainingStock() <= 0) { throw new RuntimeException("Campaign out of stock"); } quota.setUsedCount(quota.getUsedCount() + 1); quotaRepository.save(quota); campaign.setRemainingStock(campaign.getRemainingStock() - 1); campaignRepository.save(campaign); Redemption redemption = new Redemption(voucher.getId(), campaign.getId(), userId, new Date()); redemptionRepository.save(redemption); try { auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId); } catch (Exception e) { log.error("audit call failed"); } return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock()); } correct me any thing wrong

Response: Your current code has a few critical bugs and logical flaws regarding the **order of operations**, **concurrent initialization of the quota table**, and **exception handling**.

Here is what is wrong and how to fix it:

What is Wrong in Your Code?

1. **Mutating State Before Validating (Order of Operations):** You are calling `voucherRepository.claimVoucher(...)` **before** checking if the campaign is active, out of stock, or if the user has reached their limit. While `@Transactional` will roll this back if an exception is thrown, updating the database status before validation is poor practice and causes unnecessary database locking overhead.
2. **First-Time User Quota Race Condition (Duplicate Key Crash):** When a user makes their **first-ever** redemption, the `user_campaign_quota` row doesn't exist yet. `findByUserIdAndCampaignIdWithLock` returns empty, and `orElseGet` tries to `save()` a new row. If two pods make this request at the exact same millisecond for a new user, **both** will try to insert, and one will crash with a `DataIntegrityViolationException` (Duplicate Primary Key).
3. **Inconsistent Error Responses:** For some checks, you return `RedeemResponse.fail(...)`, but for others (like campaign active/stock), you throw `RuntimeException`. Unless you have a global exception handler mapping `RuntimeException` to a clean JSON response, throwing it will result in a messy 500 Internal Server Error instead of a clean failure message.

The Corrected Code

Here is the fixed, production-ready version of your method:

```
@Transactional
public RedeemResponse redeem(String code, String userId) {

    // 1. Find voucher first (basic existence check)
    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) {
        return RedeemResponse.fail("Voucher not found");
    }

    // 2. Lock and fetch the campaign FIRST to validate state and stock
    Campaign campaign = campaignRepository.findByIdWithLock(voucher.getCampaignId())
            .orElseThrow(() -> new RuntimeException("Campaign not found"));

    if (!campaign.isActive()) {
        return RedeemResponse.fail("Campaign is not active");
    }

    if (campaign.getRemainingStock() <= 0) {
        return RedeemResponse.fail("Campaign out of stock");
    }

    // 3. Lock and fetch (or initialize) user quota safely
    // Note: To prevent race conditions on first insert, ensure you handle or pre-create quotas,
    // or use a database UPSERT. Here we handle the potential duplicate key gracefully if concurrent.
    UserCampaignQuota quota;
    try {
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseGet(() -> quotaRepository.save(new UserCampaignQuota(userId, campaign.getId(), 0)));
    } catch (org.springframework.dao.DataIntegrityViolationException e) {
        // Fallback if another thread inserted the quota row concurrently just now
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseThrow(() -> new RuntimeException("Failed to initialize user quota"));
    }

    // 4. Check user limit
    if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) {
        return RedeemResponse.fail("User limit reached for this campaign");
    }

    // 5. ATOMIC CLAIM: Try to claim the voucher now that all validations passed
    int updatedRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
    if (updatedRows == 0) {
        return RedeemResponse.fail("Voucher has already been redeemed or is invalid");
```

```
    }

    // 6. Update quota and campaign stock
    quota.setUsedCount(quota.getUsedCount() + 1);
    quotaRepository.save(quota);

    campaign.setRemainingStock(campaign.getRemainingStock() - 1);
    campaignRepository.save(campaign);

    // 7. Save redemption history
    Redemption redemption = new Redemption(
            voucher.getId(), campaign.getId(), userId, new Date());
    redemptionRepository.save(redemption);

    // 8. Async/Safe audit call
    try {
        auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId);
    } catch (Exception e) {
        log.error("audit call failed", e);
    }

    return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock());
}
```

Key Improvements Made:

- **Validated first, claimed second:** We verify campaign status, stock, and quotas *before* mutating the voucher state.
- **Handled First-Time Quota Insert Clash:** Added a `try-catch` block for `DataIntegrityViolationException` around the quota lookup/creation so concurrent first-time users won't crash with a 500 error.
- **Unified Response Handling:** Standardized business logic checks to return `RedeemResponse.fail(...)` instead of throwing raw `RuntimeException`s.

User prompt: // 2. Lock and fetch the campaign FIRST to validate state and stock Campaign campaign = campaignRepository.findByIdWithLock(voucher.getCampaignId()) .orElseThrow(() -> new RuntimeException("Campaign not found")); if (!campaign.isActive()) { return RedeemResponse.fail("Campaign is not active"); } if (campaign.getRemainingStock() <= 0) { return RedeemResponse.fail("Campaign out of stock"); } if two people use different voucher but same campaign then one lock another can't process

Response: You are **100% correct**, and that is a brilliant observation.

With `findByIdWithLock(voucher.getCampaignId())`, you are placing a **Pessimistic Write Lock (`SELECT ... FOR UPDATE`)** on the entire campaign row.

If 50 people try to redeem 50 *different* vouchers under the **same campaign** at the exact same millisecond, **only one will process immediately, and the other 49 will be forced to wait in line** for that row lock to release. Under heavy flash-sale traffic, this causes severe database bottlenecks, thread blocking, and potential timeout errors.

The Better Approach: Atomic Campaign Stock Decrement

Instead of locking the entire campaign row upfront, you can use a **conditional atomic update** on the campaign stock—just like we did for the voucher. This allows thousands of users to redeem different vouchers for the same campaign **in parallel** without blocking each other.

1. Add this update method to `CampaignRepository`:

```
@Modifying
@Query("UPDATE Campaign c SET c.remainingStock = c.remainingStock - 1 WHERE c.id = :campaignId AND c.remai
int decrementStock(@Param("campaignId") Long campaignId);
```

- **Why this works:** The database executes this as a single atomic operation. It checks `remainingStock > 0` and `active = true` and decrements it instantly. If stock hits 0 or the campaign is inactive, it updates `0` rows.

2. Updated Service Method (Non-Blocking for Same Campaign)

By removing `findByIdWithLock` on the campaign, users redeeming different vouchers for the same campaign can run concurrently:

```
@Transactional
public RedeemResponse redeem(String code, String userId) {

    // 1. Find voucher first
    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) {
        return RedeemResponse.fail("Voucher not found");
    }

    // 2. Quick check if campaign exists and is active (non-blocking read)
    Campaign campaign = campaignRepository.findById(voucher.getCampaignId())
            .orElseThrow(() -> new RuntimeException("Campaign not found"));

    if (!campaign.isActive()) {
        return RedeemResponse.fail("Campaign is not active");
```

```
    }

    // 3. ATOMICALLY decrement campaign stock (Prevents overselling without row locking)
    int updatedCampaignRows = campaignRepository.decrementStock(campaign.getId());
    if (updatedCampaignRows == 0) {
        return RedeemResponse.fail("Campaign out of stock or inactive");
    }

    // 4. Lock and fetch (or initialize) user quota safely (Only locks this specific user's quota, not the
    UserCampaignQuota quota;
    try {
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseGet(() -> quotaRepository.save(new UserCampaignQuota(userId, campaign.getId(), 0)));
    } catch (org.springframework.dao.DataIntegrityViolationException e) {
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseThrow(() -> new RuntimeException("Failed to initialize user quota"));
    }

    // 5. Check user limit
    if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) {
        // IMPORTANT: Because we already decremented stock and claimed the voucher,
        // if the user hits their limit, we must throw an exception to trigger a rollback!
        throw new UserLimitExceededException("User limit reached for this campaign");
    }

    // 6. ATOMIC CLAIM VOUCHER
    int updatedVoucherRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
    if (updatedVoucherRows == 0) {
        throw new RuntimeException("Voucher has already been redeemed or is invalid");
    }

    // 7. Increment user quota
    quota.setUsedCount(quota.getUsedCount() + 1);
    quotaRepository.save(quota);

    // 8. Save redemption history
    Redemption redemption = new Redemption(
            voucher.getId(), campaign.getId(), userId, new Date());
    redemptionRepository.save(redemption);

    // 9. Audit log (Async)
    try {
        auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId);
    } catch (Exception e) {
        log.error("audit call failed", e);
    }

    // Note: getRemainingStock() in response can be fetched or calculated,
    // since we decremented it in DB, you can fetch the latest campaign or return estimated stock.
    return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock() - 1);
}
```

Why this is much better for scalability:

1. **No Campaign Row Locking:** Multiple pods can process different vouchers under the *same campaign* simultaneously without waiting in line.
2. **Safe Stock Control:** The database's atomic `UPDATE` guarantees `remainingStock` will never drop below `0`, preventing race conditions and overselling.
3. **Targeted Locking:** The only locking that happens is on the individual `voucher` (via conditional update) and the individual `user_campaign_quota` (via row lock), meaning contention only happens if the *exact same user* or *exact same voucher* is requested concurrently.

User prompt: if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) { // IMPORTANT: Because we already decremented stock and claimed the voucher, // if the user hits their limit, we must throw an exception to trigger a rollback! throw newUserLimitExceededException("User limit reached for this campaign"); } if reached limit I need return a response to user

Response: You are completely right. Forcing an exception just to trigger a rollback is messy for API design. It is much cleaner to **return `RedeemResponse.fail(...)` directly** when the limit is reached.

To do this safely without needing a rollback, you can **check and lock the user quota *before* you claim the voucher or decrement the stock**.

Because you lock the quota row first, concurrent requests from the *same user* are forced to wait. The first request will pass; the second request will get the updated quota count, see that the limit is reached, and **return a failure response immediately** before any changes are made to the database.

Here is the correct order of operations that allows clean `RedeemResponse.fail(...)` returns:

The Clean, Early-Exit Implementation

```
@Transactional
public RedeemResponse redeem(String code, String userId) {

    // 1. Find voucher
    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) {
        return RedeemResponse.fail("Voucher not found");
    }

    // 2. Find campaign & check basic state (non-blocking)
    Campaign campaign = campaignRepository.findById(voucher.getCampaignId())
            .orElseThrow(() -> new RuntimeException("Campaign not found"));

    if (!campaign.isActive()) {
        return RedeemResponse.fail("Campaign is not active");
    }

    if (campaign.getRemainingStock() <= 0) {
        return RedeemResponse.fail("Campaign out of stock");
    }

    // 3. Lock and check user quota FIRST (Before making any mutations)
    UserCampaignQuota quota;
    try {
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseGet(() -> quotaRepository.save(new UserCampaignQuota(userId, campaign.getId(), 0)));
    } catch (org.springframework.dao.DataIntegrityViolationException e) {
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseThrow(() -> new RuntimeException("Failed to initialize user quota"));
    }

    // 4. CHECK LIMIT EARLY: Return clean response immediately if limit reached.
    // Because no stock or vouchers have been modified yet, we don't need a rollback!
    if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) {
        return RedeemResponse.fail("User limit reached for this campaign");
    }

    // 5. ATOMICALLY decrement campaign stock
    int updatedCampaignRows = campaignRepository.decrementStock(campaign.getId());
    if (updatedCampaignRows == 0) {
        return RedeemResponse.fail("Campaign out of stock");
    }

    // 6. ATOMIC CLAIM VOUCHER
    int updatedVoucherRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
    if (updatedVoucherRows == 0) {
        // If voucher claim fails here, we must throw an exception to roll back the stock decrement
        throw new RuntimeException("Voucher has already been redeemed or is invalid");
    }

    // 7. Increment user quota and save
    quota.setUsedCount(quota.getUsedCount() + 1);
    quotaRepository.save(quota);

    // 8. Save redemption history
    Redemption redemption = new Redemption(
            voucher.getId(), campaign.getId(), userId, new Date());
    redemptionRepository.save(redemption);

    // 9. Audit log (Async/Safe)
    try {
        auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId);
    } catch (Exception e) {
        log.error("audit call failed", e);
    }

    return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock() - 1);
}
```

Why this flow works perfectly:

1. **No Mutations on Quota Exceeded:** If Step 4 catches that the user reached their limit, it returns `RedeemResponse.fail(...)` instantly. No stock was taken, no voucher was claimed, and no transaction rollback is required.
2. **Race-Condition Safe for Users:** Because of `findByUserIdAndCampaignIdWithLock` at Step 3, concurrent requests from the *same user* are serialized. User A gets in, increments the quota, and finishes. User B waits, reads the newly incremented quota, hits Step 4, and gets a clean failure response.

User prompt: // 6. ATOMIC CLAIM VOUCHER int updatedVoucherRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date()); if (updatedVoucherRows == 0) { // If voucher claim fails here, we must throw an exception to roll back the stock decrement throw new RuntimeException("Voucher has already been redeemed or is invalid"); } redeemed or void need return reponse

Response: You caught a very important detail. If you just return `RedeemResponse.fail(...)` instead of throwing an exception, **Spring will not roll back the transaction**, leaving the campaign stock decremented even though the voucher was already redeemed or void.

To fix this cleanly and return an exact error response (like *"Voucher already redeemed"* or *"Voucher is void"*) **without needing a rollback**, you simply need to **change the order of operations**:

1. Check quota & limits first.
2. **Atomically claim the voucher first.** If it fails, return a failure response *immediately* (no changes have been made yet, so no rollback needed).
3. **Decrement campaign stock second.** If *this* fails (out of stock), throw an exception to roll back the voucher claim.

The Corrected Implementation

```
@Transactional
public RedeemResponse redeem(String code, String userId) {

    // 1. Find voucher
    Voucher voucher = voucherRepository.findByCode(code);
    if (voucher == null) {
        return RedeemResponse.fail("Voucher not found");
    }

    // 2. Find campaign (needed for limit check)
    Campaign campaign = campaignRepository.findById(voucher.getCampaignId())
            .orElseThrow(() -> new RuntimeException("Campaign not found"));

    if (!campaign.isActive()) {
        return RedeemResponse.fail("Campaign is not active");
    }

    // 3. Lock and check user quota first
    UserCampaignQuota quota;
    try {
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseGet(() -> quotaRepository.save(new UserCampaignQuota(userId, campaign.getId(), 0)));
    } catch (org.springframework.dao.DataIntegrityViolationException e) {
        quota = quotaRepository.findByUserIdAndCampaignIdWithLock(userId, campaign.getId())
            .orElseThrow(() -> new RuntimeException("Failed to initialize user quota"));
    }

    if (quota.getUsedCount() >= campaign.getUserVoucherLimit()) {
        return RedeemResponse.fail("User limit reached for this campaign");
    }

    // 4. ATOMIC CLAIM VOUCHER FIRST
    int updatedVoucherRows = voucherRepository.claimVoucher(voucher.getId(), userId, new Date());
    if (updatedVoucherRows == 0) {
        // Since NO stock or quota has been changed yet, we can return a clean response
        // directly WITHOUT needing a rollback! Re-fetch to see why it failed.
        Voucher currentVoucher = voucherRepository.findById(voucher.getId()).orElse(voucher);

        if ("REDEEMED".equals(currentVoucher.getStatus())) {
            return RedeemResponse.fail("Voucher has already been redeemed");
        } else if ("VOID".equals(currentVoucher.getStatus())) {
            return RedeemResponse.fail("Voucher is void");
        }
        return RedeemResponse.fail("Voucher is invalid");
    }

    // 5. ATOMICALLY DECREMENT CAMPAIGN STOCK SECOND
    int updatedCampaignRows = campaignRepository.decrementStock(campaign.getId());
    if (updatedCampaignRows == 0) {
        // If stock ran out at the exact same millisecond, throw an exception
        // to automatically roll back the voucher claim we just did!
        throw new RuntimeException("Campaign out of stock");
    }

    // 6. Increment user quota and save
    quota.setUsedCount(quota.getUsedCount() + 1);
    quotaRepository.save(quota);

    // 7. Save redemption history
    Redemption redemption = new Redemption(
            voucher.getId(), campaign.getId(), userId, new Date());
    redemptionRepository.save(redemption);

    // 8. Audit log (Async/Safe)
    try {
        auditClient.recordRedemption(campaign.getClientCode(), voucher.getCode(), userId);
    } catch (Exception e) {
        log.error("audit call failed", e);
    }
```

```
    return RedeemResponse.ok(voucher.getCode(), campaign.getRemainingStock() - 1);
}
```

Why this structure is ideal:

- **No Rollback Needed for Voucher Failures:** If a voucher is already redeemed or void, Step 4 catches it instantly and returns a clean API response before touching stock or quotas.
- **Accurate Error Messages:** By re-fetching the voucher status when `updatedVoucherRows == 0`, you can precisely tell the user whether it was already redeemed or void.
- **Safe Stock Rollbacks:** If the campaign runs out of stock right as someone claims the last voucher, throwing an exception at Step 5 safely rolls back the voucher claim.

User prompt: anything can improve in future. add reddis

Response: Adding **Redis** is the ultimate future-proofing step for this architecture, especially if you anticipate traffic spikes or flash sales where thousands of users hit your multi-pod setup simultaneously. Database row locks work well, but they can still bottleneck under extreme loads.

Here is how you can leverage Redis in the future to make your redemption flow blazing fast and scalable:

1. Redis Atomic Stock Decrement (Flash Sale Mode)

Instead of hitting the database to check and decrement `remaining_stock`, you can store the campaign's remaining stock in Redis as an atomic counter.

- **How it works:** When a request comes in, you call Redis `DECRBY campaign:{id}:stock 1`.
- **The Benefit:** Redis handles tens of thousands of concurrent requests in-memory without putting any load on your SQL database. If the returned value is `< 0`, you instantly reject the request as "Out of stock." You can then sync the stock back to the database asynchronously or periodically.

2. Redis-Based User Quota Tracking & Rate Limiting

Instead of using the `user_campaign_quota` table with pessimistic locks, you can track user redemptions in Redis using atomic increments.

- **How it works:** Use a Redis key like `campaign:{campaignId}:user:{userId}:count`.
- When a user tries to redeem, you use `INCR` (or check the count first). If it exceeds `userVoucherLimit`, you reject it instantly in memory.
- **The Benefit:** Zero database contention for checking user limits.

3. Distributed Locking (Redisson) for Critical Sections

If you still want to write the final state to your relational database safely, you can use **Redisson (Redis Distributed Locks)** to wrap the transaction per user or per voucher.

```
String lockKey = "lock:redeem:" + campaign.getId() + ":" + userId;
RLock lock = redissonClient.getLock(lockKey);

if (lock.tryLock(2, 5, TimeUnit.SECONDS)) {
    try {
        // Execute your database transaction safely here knowing
        // no other pod is processing this user at the same time.
    } finally {
        lock.unlock();
    }
}
```

Future Architecture Blueprint

1. **Fast-Fail Layer (Redis):** Check if the campaign is active/in-stock and if the user has hit their quota entirely in Redis memory (< 2ms response time).
2. **Atomic Claim Layer (DB):** Use your existing `claimVoucher()` query to ensure the specific voucher code isn't double-spent.
3. **Async Processing:** Push successful redemptions to a message queue (like RabbitMQ or Kafka) to handle database inserts, history logging, and third-party audit calls asynchronously.

User prompt: @Repository public interface QuotaRepository extends JpaRepository<UserCampaignQuota, UserCampaignQuotaId> { @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("SELECT q FROM UserCampaignQuota q WHERE q.userId = :userId AND q.campaignId = :campaignId") Optional<UserCampaignQuota> findByUserIdAndCampaignIdWithLock(@Param("userId") String userId, @Param("campaignId") Long campaignId); } why here need composite key

Response: A composite key is needed because the primary key of your `user_campaign_quota` table is made of **two columns combined**: `user_id` and `campaign_id`.

Here is why this structure is required both logically and technically:

1. The Business Logic (Why a single ID doesn't work)

- Can `user_id` be the primary key by itself? **No**, because a single user can participate in *multiple* different campaigns.
- Can `campaign_id` be the primary key by itself? **No**, because a single campaign has *many* different users.

The only way to uniquely identify a single quota record is by combining **both**: *Which user* in *which campaign*? Therefore, the unique identity is the pair `(user_id, campaign_id)`.

2. The Database Level

In your database table, you defined a composite primary key:

```
PRIMARY KEY (user_id, campaign_id)
```

This ensures that a user can only have **one** quota row per campaign, preventing duplicate rows for the same user-campaign pair.

3. The JPA Requirement

Java Persistence API (JPA) requires every entity to have an `@Id`.

- If a table has a single primary key column (like an auto-incrementing `id`), you just use `@Id private Long id;`.
- When a table has **two or more** columns making up the primary key, JPA cannot use a standard single `@Id`. It requires an **`@IdClass`** (like `UserCampaignQuotaId`) so it knows how to bundle those two fields together when executing methods like `findById()`.

Alternative (Surrogate Key)

If you didn't want to use a composite key, you could give the table its own auto-incrementing `id` (e.g., `BIGINT AUTO_INCREMENT PRIMARY KEY`) and add a **Unique Constraint** on `(user_id, campaign_id)`. However, using a composite key is cleaner here because a user-campaign relationship naturally relies on those two values as its true identity.

User prompt: public class UserCampaignQuotaId implements Serializable { private String userId; private Long campaignId; public UserCampaignQuotaId() {} public UserCampaignQuotaId(String userId, Long campaignId) { this.userId = userId; this.campaignId = campaignId; } @Override public boolean equals(Object o) { if (this == o) return true; if (o == null || getClass() != o.getClass()) return false; UserCampaignQuotaId that = (UserCampaignQuotaId) o; return Objects.equals(userId, that.userId) && Objects.equals(campaignId, that.campaignId); } @Override public int hashCode() { return Objects.hash(userId, campaignId); } } this class for what

Response: The `UserCampaignQuotaId` class is an **Identity Class** required by JPA when an entity has a **composite primary key** (a primary key made up of two or more columns combined).

Since your `user_campaign_quota` table uses both `user_id` and `campaign_id` together as its primary key (`PRIMARY KEY (user_id, campaign_id)`), JPA needs a way to wrap those two fields into a single object.

Here is a breakdown of why each part of this class is necessary:

1. Why `Serializable`?

```
public class UserCampaignQuotaId implements Serializable
```

- **The JPA Requirement:** The JPA specification explicitly states that any composite primary key class **must** implement `Serializable`. This allows Java to serialize the key object when caching, clustering, or passing it through persistence contexts.

2. Why the fields match the Entity (`userId` & `campaignId`)?

```
private String userId;
private Long campaignId;
```

- **The Mapping:** JPA requires the field names and data types in this ID class to **vividly match** the fields marked with `@Id` in your main `UserCampaignQuota` entity. This is how JPA knows which database columns map to which parts of the composite key.

3. Why `equals()` and `hashCode()` are mandatory?

```
@Override
public boolean equals(Object o) { ... }

@Override
public int hashCode() { ... }
```

- **Unique Identification:** JPA uses `equals()` and `hashCode()` under the hood to track entities in its internal persistence cache, compare object states, and execute queries like `quotaRepository.findById(id)`.
- If you don't override these methods, Java will compare memory addresses instead of the actual string (`userId`) and number (`campaignId`), causing JPA to fail when trying to look up or save records.

User prompt: // 5. ATOMICALLY DECREMENT CAMPAIGN STOCK SECOND int updatedCampaignRows = campaignRepository.decrementStock(campaign.getId()); if (updatedCampaignRows == 0) { // If stock ran out at the exact same millisecond, throw an exception // to automatically roll back the voucher claim we just did! throw new RuntimeException("Campaign out of stock"); } user need get response campaign out stock

Response: To return a clean failure response (like `RedeemResponse.fail("Campaign out of stock")`) instead of a 500 error, while **still rolling back the voucher claim**, you can manually tell Spring to roll back the transaction before returning.

By default, Spring only rolls back transactions when an unchecked exception is thrown. If you just return a response object, Spring thinks everything succeeded and **commits** the database changes.

You can solve this cleanly using `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();`:

The Solution: Manual Rollback + Clean Response

```
// 5. ATOMICALLY DECREMENT CAMPAIGN STOCK SECOND
int updatedCampaignRows = campaignRepository.decrementStock(campaign.getId());
if (updatedCampaignRows == 0) {
    // Explicitly tell Spring to roll back the voucher claim & quota increment we just did!
    org.springframework.transaction.interceptor.TransactionAspectSupport
        .currentTransactionStatus()
        .setRollbackOnly();

    // Now we can safely return a clean response to the user
    return RedeemResponse.fail("Campaign out of stock");
}
```

Why this works:

1. **Rolls Back Data:** `setRollbackOnly()` ensures that the voucher claim and quota update are undone in the database so data consistency is preserved.
2. **Clean API Response:** Instead of crashing with a Java exception, the controller immediately receives `RedeemResponse.fail("Campaign out of stock")` and sends a proper JSON response back to the user.
